

COMPUTER SCIENCE & ENGINEERING — OBJECT-ORIENTED PROGRAMMING

Airline Baggage & Boarding Pass Fee Engine

A Comprehensive Problem Statement on C++ Abstract Classes, Pure Virtual Functions, Dynamic Memory Allocation, and Runtime Polymorphism

Key Learning Objectives:

- Define and enforce an Abstract Base Class (FlightTicket) with Pure Virtual Functions (= 0).
- Model fare computation, baggage penalties, and service surcharges across multiple flight tiers.
- Manage dynamic runtime collections using raw double pointers (FlightTicket**) and heap allocation (new[]).
- Ensure safe dynamic deallocation and prevent memory leaks using virtual destructors and delete[] loops.
- Implement runtime polymorphism to generate boarding passes and calculate aggregate flight revenue.

1. Problem Background & Domain Overview

Global airline reservation platforms must dynamically process ticketing, ancillary fee calculations, and boarding pass generation for thousands of passengers every day. The ticketing system serves three distinct fare tiers: Economy Ticket, Business Ticket, and First Class Ticket.

While every reservation shares basic passenger identification and core flight numbers, baggage allowances, excess weight charges, seat selection fees, and complimentary luxury amenities vary drastically across travel classes.

In this assignment, you will build the core billing and boarding pass engine in C++. Because flight manifests vary in size, the total number of passengers (N) is unknown at compile time and must be read at runtime using dynamic heap allocation (FlightTicket**).

2. Object-Oriented Class Specifications

2.1 Abstract Base Class: FlightTicket

The base class acts as an abstract contract that cannot be directly instantiated.

- **Protected Attribute:** string passengerName
- **Protected Attribute:** string flightNumber (e.g., 'AA-402', 'BA-117')
- **Protected Attribute:** double baseFare (Base ticket price in USD)
- **Constructor:** FlightTicket(string name, string flightNo, double fare) to initialize common data.
- **Virtual Destructor:** virtual ~FlightTicket() — Virtual destructor to ensure derived class objects are destroyed cleanly through base pointers.
- **Pure Virtual Method 1:** virtual double calculateTotalFare(int baggageCount, double totalWeightKg) = 0 — Computes final payable amount including all baggage penalties and surcharges.

- **Pure Virtual Method 2:** virtual void printBoardingPass() = 0 — Polymorphically prints passenger details, seat category, baggage fees, and the final ticket price.

2.2 Concrete Derived Class: EconomyTicket

- **Private Attribute:** double extraBagFeePerKg (Charge per kg over the standard 15 kg allowance)
- **Private Attribute:** double seatSelectionFee (Optional seat reservation surcharge)
- **Fare Logic:** Allowance: 1 free bag up to 15 kg. Excess weight is charged at (totalWeight - 15) * extraBagFeePerKg. Extra bags beyond 1 incur a flat surcharge of \$50 per bag.
- **Total Formula:** Total Fare = baseFare + seatSelectionFee + excessBaggageCharges.

2.3 Concrete Derived Class: BusinessTicket

- **Private Attribute:** double loungeAccessFee (Fixed VIP lounge tariff)
- **Private Attribute:** double priorityBoardingFee (Fast-track security and boarding fee)
- **Fare Logic:** Allowance: 2 free bags up to a combined 32 kg. Excess weight above 32 kg is billed at a flat rate of \$20 per kg.
- **Total Formula:** Total Fare = baseFare + loungeAccessFee + priorityBoardingFee + excessWeightCharges.

2.4 Concrete Derived Class: FirstClassTicket

- **Private Attribute:** double conciergeServiceFee (Personal terminal concierge and private transport fee)
- **Fare Logic:** Allowance: Up to 3 complimentary bags with no weight penalties up to 50 kg total.
- **Penalty Logic:** A flat penalty of \$100 is charged only if total baggage weight exceeds 50 kg.
- **Total Formula:** Total Fare = baseFare + conciergeServiceFee + flatOversizePenalty.

3. Class Architecture Summary

Class Name	Inheritance	Specific Data Members	Baggage / Fee Rules
EconomyTicket	public FlightTicket	extraBagFeePerKg, seatSelectionFee	1 bag (15kg free), \$50/extra bag + per-kg excess
BusinessTicket	public FlightTicket	loungeAccessFee, priorityBoardingFee	2 bags (32kg free), \$20/kg for weight > 32kg
FirstClassTicket	public FlightTicket	conciergeServiceFee	3 bags (50kg free), flat \$100 penalty if > 50kg

4. Dynamic Memory & Main Function Workflow

The application must manage passenger bookings dynamically using raw pointers on the heap without hardcoded array limits:

- 1. Dynamic Capacity: Prompt the airport ticketing agent for the total passenger count (N).
- 2. Pointer Array Allocation: Allocate a dynamic array of base pointers on the heap:

```
int N;
cout << "Enter number of passenger bookings to process: ";
cin >> N;

// Allocate dynamic array of base pointers on heap
FlightTicket** manifest = new FlightTicket*[N];
```

- 3. Factory Input Loop: Loop from index 0 to N - 1. Prompt for passenger name, flight number, base fare, baggage count, total weight, and ticket category (1: Economy, 2: Business, 3: First Class), allocating the appropriate derived object dynamically.
- 4. Polymorphic Processing: Iterate through the array using base class pointers manifest[i] to invoke calculateTotalFare() and printBoardingPass().
- 5. Flight Revenue Aggregation: Accumulate and print the total revenue collected across all passengers on the manifest.
- 6. Clean Heap Deallocation: Deallocate every passenger instance and free the pointer array:

```
// Clean deallocation
for (int i = 0; i < N; ++i) {
    delete manifest[i]; // Triggers virtual destructor
}
delete[] manifest;      // Frees the pointer array
manifest = nullptr;
```

5. Sample Program Execution & I/O Trace

```
=== GLOBAL AIRLINES RESERVATION & BAGGAGE ENGINE ===
Enter number of passenger bookings to process: 3

--- Passenger #1 Entry ---
Select Fare Class (1: Economy, 2: Business, 3: First Class): 1
Enter Passenger Name: John Miller
Enter Flight Number: AI-204
Enter Base Fare ($): 350.00
Enter Baggage Count: 2
Enter Total Baggage Weight (kg): 22.0
Enter Extra Bag Fee Rate ($/kg): 15.00
Enter Seat Selection Fee ($): 25.00

--- Passenger #2 Entry ---
Select Fare Class (1: Economy, 2: Business, 3: First Class): 2
Enter Passenger Name: Sarah Jenkins
Enter Flight Number: AI-204
Enter Base Fare ($): 950.00
Enter Baggage Count: 2
Enter Total Baggage Weight (kg): 36.0
Enter Lounge Access Fee ($): 75.00
```

```

Enter Priority Boarding Fee ($): 40.00

--- Passenger #3 Entry ---
Select Fare Class (1: Economy, 2: Business, 3: First Class): 3
Enter Passenger Name: Lord Alexander
Enter Flight Number: AI-204
Enter Base Fare ($): 2400.00
Enter Baggage Count: 3
Enter Total Baggage Weight (kg): 48.0
Enter Concierge Service Fee ($): 200.00

===== FLIGHT BOARDING MANIFEST =====
[BOARDING PASS - ECONOMY CLASS]
  Passenger      : John Miller
  Flight Number  : AI-204
  Base Fare      : $350.00
  Seat Surcharge : $25.00
  Baggage Charge : $155.00 (1 Extra Bag: $50 + 7.0kg Excess: $105.00)
  TOTAL FARE     : $530.00
-----
[BOARDING PASS - BUSINESS CLASS]
  Passenger      : Sarah Jenkins
  Flight Number  : AI-204
  Base Fare      : $950.00
  VIP Amenities  : Lounge ($75.00) + Priority Boarding ($40.00)
  Baggage Charge : $80.00 (4.0kg Excess @ $20/kg)
  TOTAL FARE     : $1,145.00
-----
[BOARDING PASS - FIRST CLASS]
  Passenger      : Lord Alexander
  Flight Number  : AI-204
  Base Fare      : $2,400.00
  Concierge Fee  : $200.00
  Baggage Charge : $0.00 (Within 50kg Complimentary Allowance)
  TOTAL FARE     : $2,600.00
=====
FLIGHT MANIFEST REVENUE SUMMARY:
  Total Passengers Checked In : 3
  Total Revenue Collected    : $4,275.00
=====
Memory cleaned up successfully. 3 passenger objects deallocated.

```